Self-supervised Learning
Clustering

Hermann Blum

UNIVERSITÄT BONN

LAMARR INSTITUTE FOR MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Robot Perception and Learning Lab
rpl.uni-bonn.de

Today

- Nearest Neighbors
- Clustering
- Deep Learning based Hybrids

k Nearest Neighbors

- Given a set of labelled training data and a sample x:
  - find nearest neighbors in the training data
  - output the majority class of the k-neighborhood

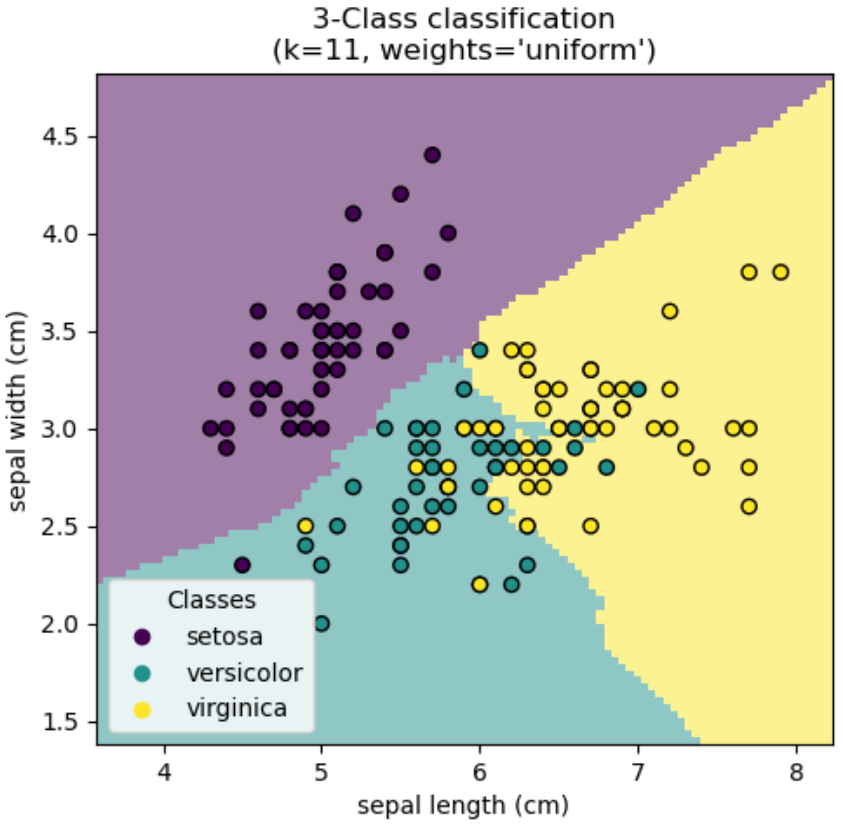
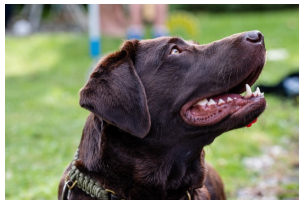


image: Scikit-Learn

How do I find the nearest neighbors?



$\{a_1, a_2, a_3, \dots, a_N\}$



$\{b_1, b_2, b_3, \dots, b_N\}$



$\{c_1, c_2, c_3, \dots, c_N\}$

Euclidean Distance $d = \sqrt{(a-b)^2}$

Cosine Distance $d = \frac{a \cdot b}{|a| \cdot |b|}$

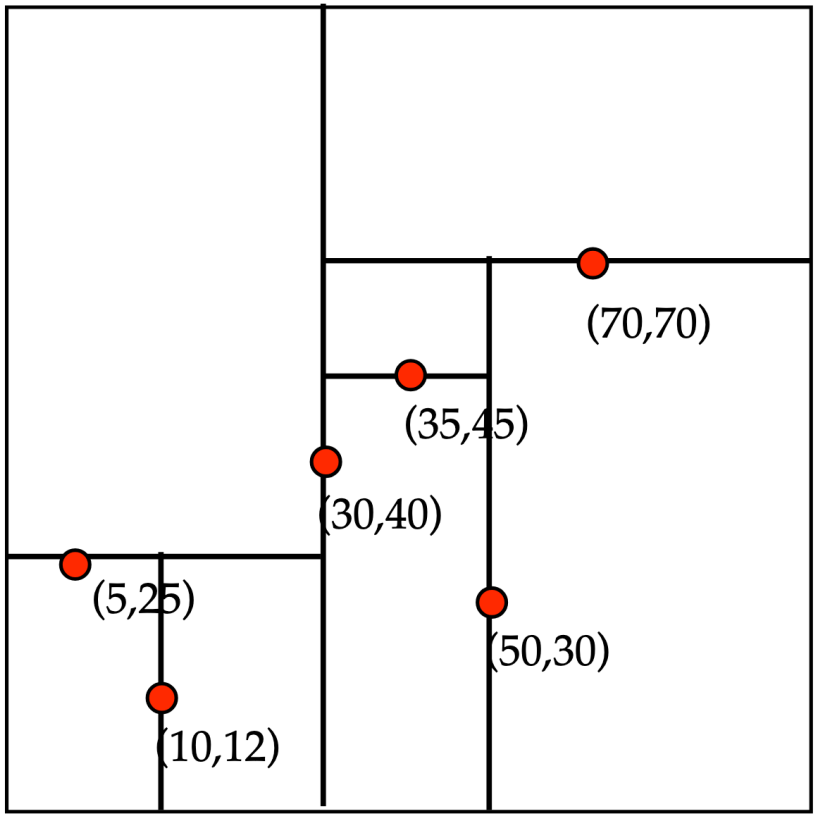
Which distance metric should I use?

- Cosine Similarity is invariant to the vector magnitude, it measures only the angle between vectors
- In practice, papers often find little difference for high-dimensional descriptors -> features are often well normalized and spread over a hypersphere
- For a specific embedding, check how the network is trained: Does the loss optimize a distance? which is it?
- DINO descriptors are compared by cosine similarity

How can I efficiently find nearest neighbors?

KD-Trees:

insert: (30,40), (5,25), (10,12), (70,70), (50,30), (35,45)



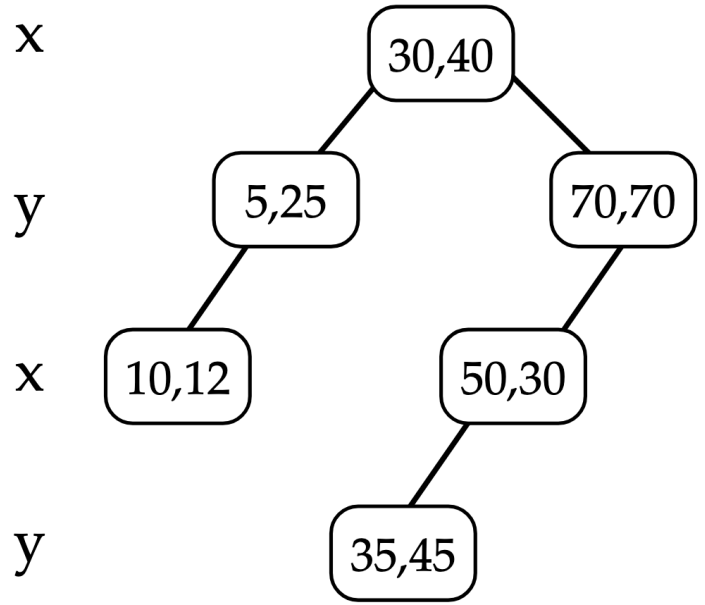
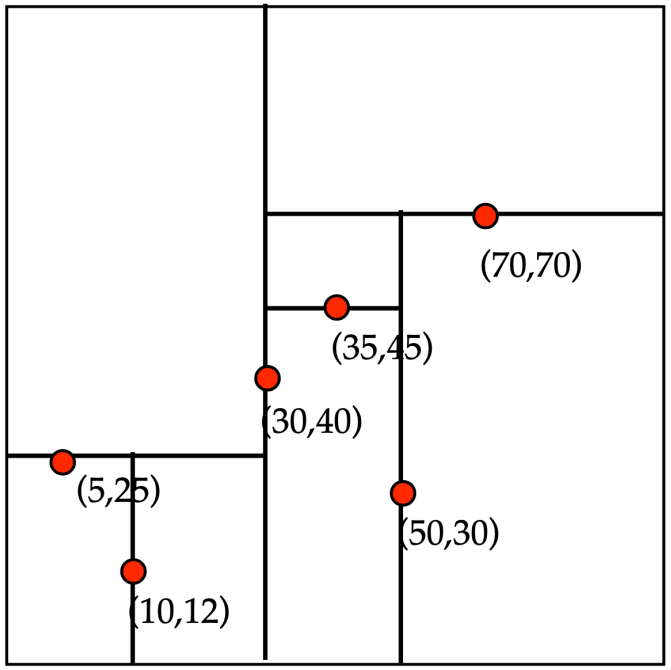


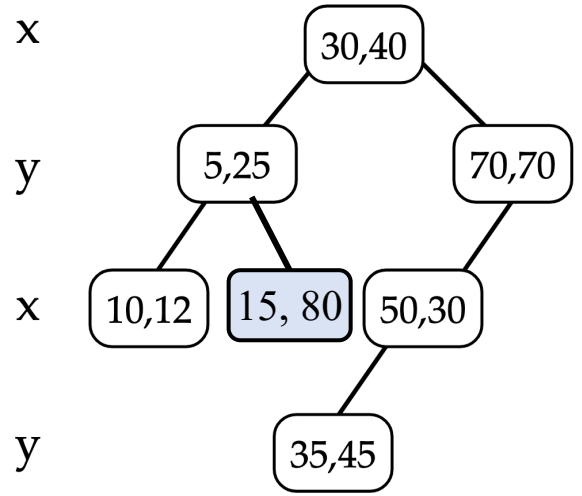
Illustration by Carl Kingsford, CMU

Insert Nodes in KD-Trees

1. Begin at the root and traverse the tree until you arrive at a leaf
2. Insert the new node

Example: (15, 80)



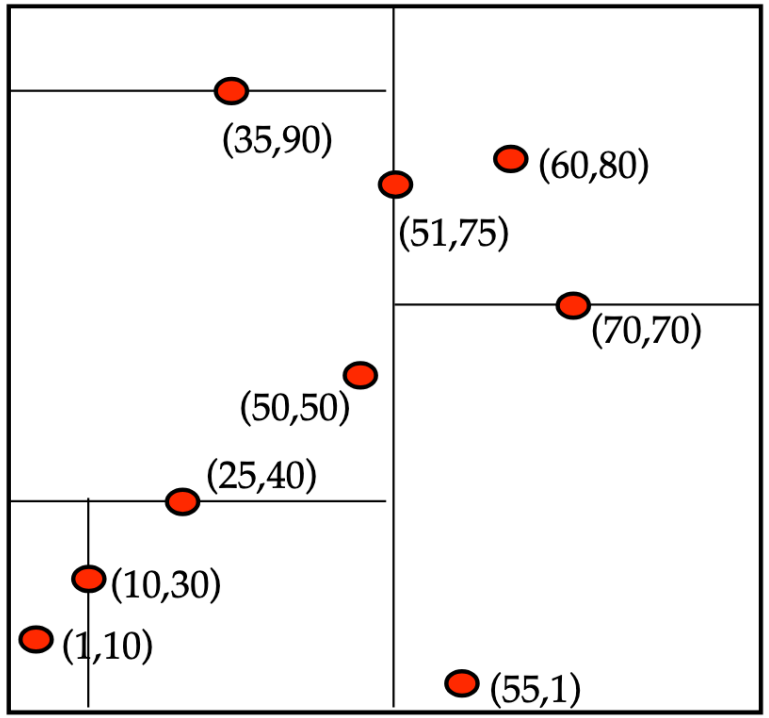


Nearest Neighbors in KD-Trees

1. initialize `best` and `best_dist`
2. traverse tree like for insertion
   1. for each visited node, compute `current_dist` and compare to `best_dist`
   2. if distance is smaller, `best_dist = current_dist`
3. upon leafe, unwind the search path
   1. at each junction, could there be closer nodes on the other side?
      i.e. is the deviation to the splitting value smaller than `best_dist`?
   2. if yes, traverse other side

Nearest Neighbors in KD-Trees

Example: Search NN for (52,52)



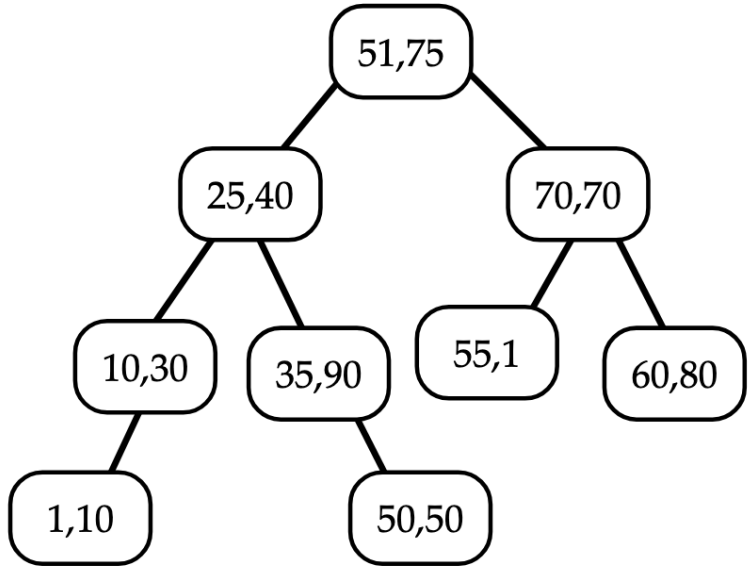


Illustration by Carl Kingsford, CMU

Locality Sensitive Hashing (LSH)

h is a locality sensitive hash function for a distance function d

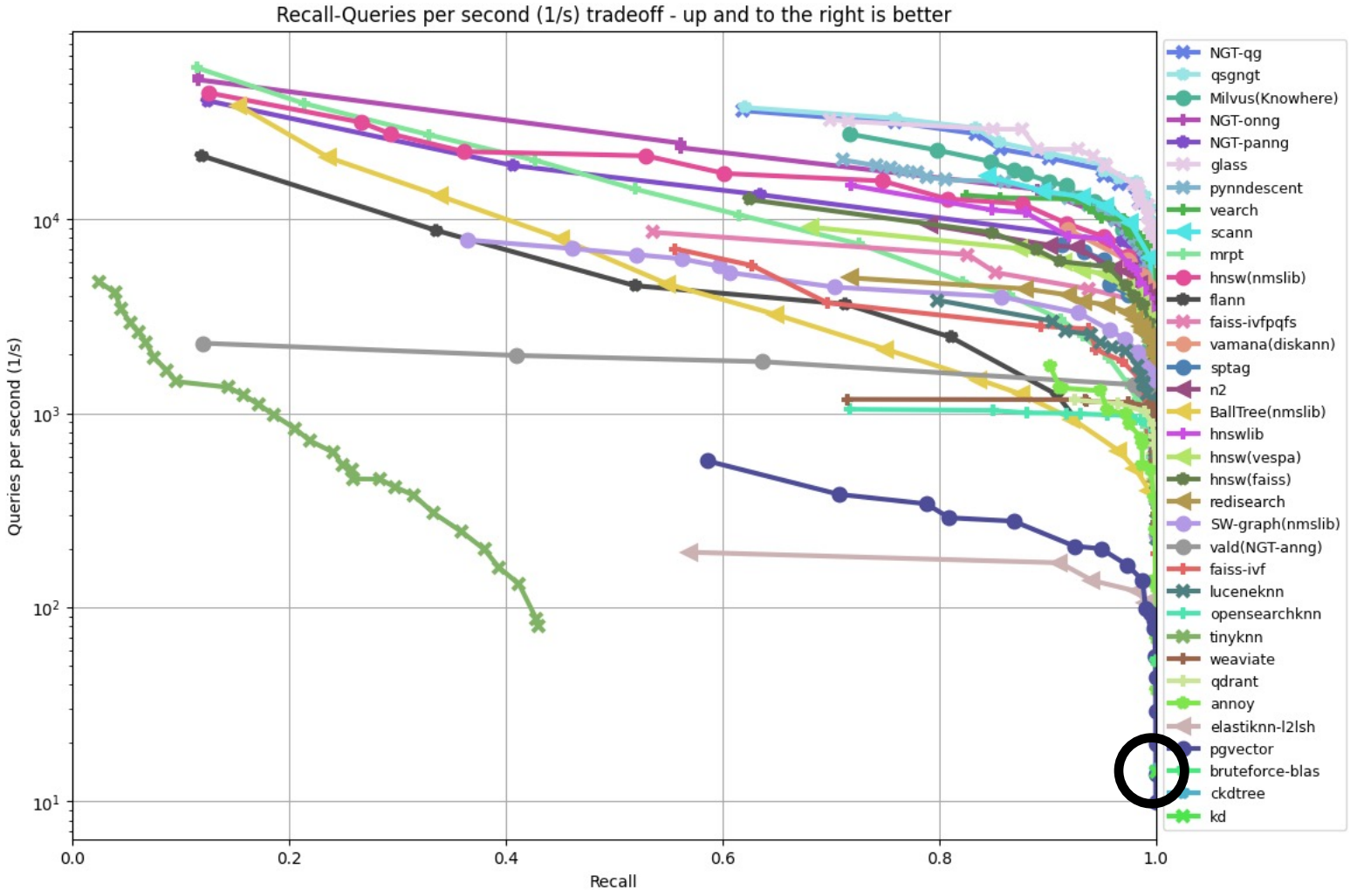
if $d(a,b) \leq r$ then $p[h(a) = h(b)] \geq p_1$

and if $d(a,b) \geq c \cdot r$ then $p[h(a) = h(b)] \leq p_2$

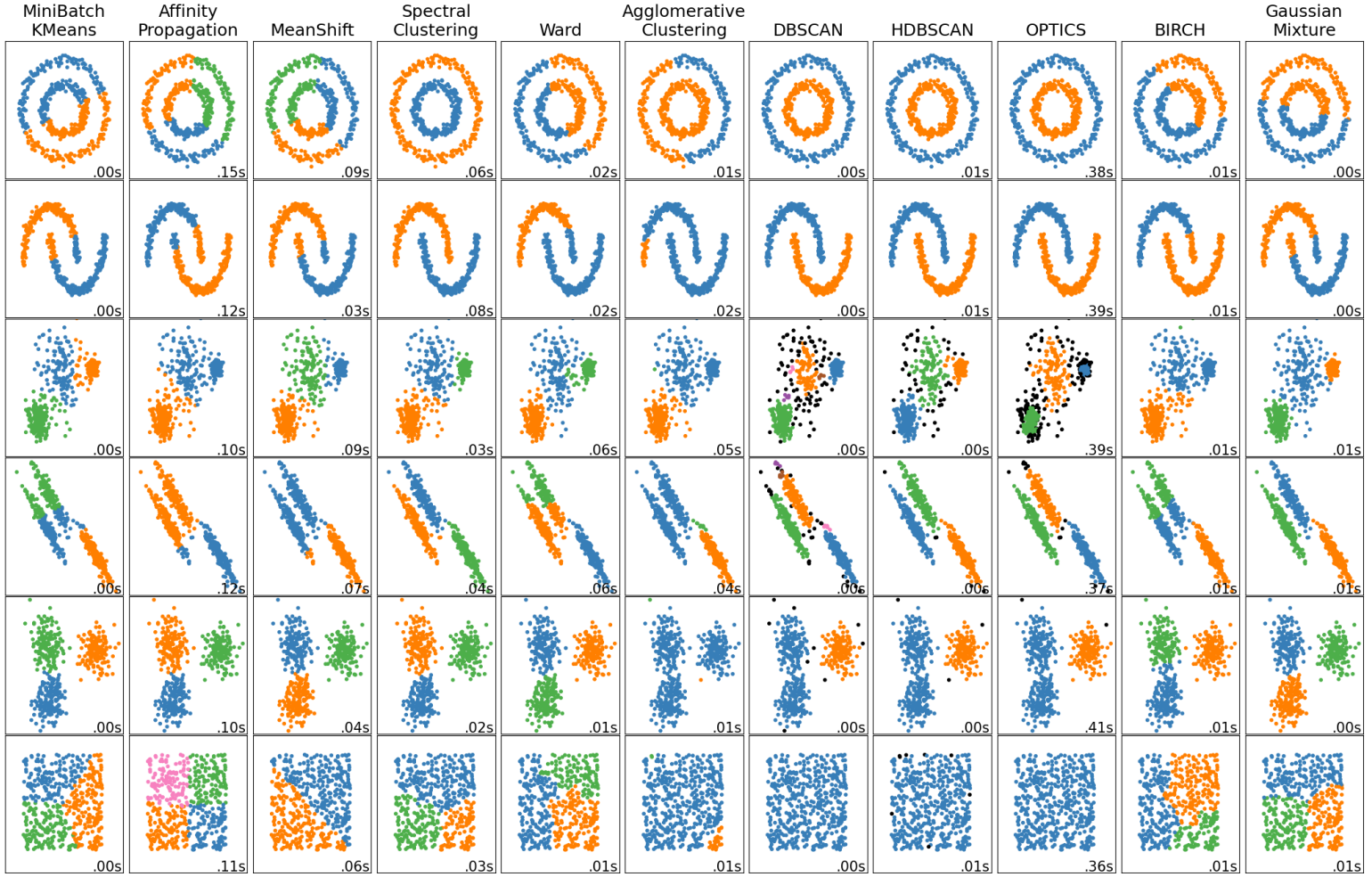
Approximate Nearest Neighbor Search through LSH

1. Sort data into L hash tables, each table based on a different, randomly constructed LSH function
2. For query q, compute $h_i(q)$ for the i-th hash table and compute distances to all points in the corresponding bucket
3. Iterate through the hash tables until a point within stop margin is found, or no hash tables are left.

ann-benchmarks.com



Unsupervised Classification: Clustering



Unsupervised Classification: Clustering

There is no perfect clustering algorithm. The choice often comes down to problem parameters like:

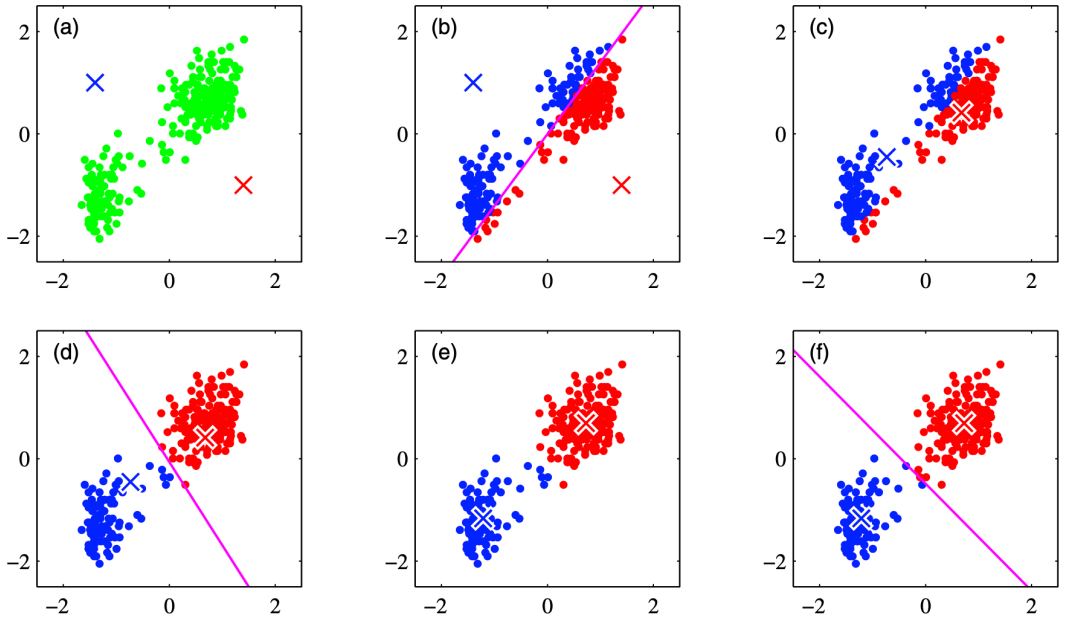
- Do I know the number of clusters or not?
- How complex is the problem?
  - Does all my data fit into RAM?
  - How much time do I have?

k Means

1. Initialize k vectors μ_i
2. E-Step: Assign each data sample to the closest μ_i, resulting in sets X_i
3. M-Step: Recompute

$$\mu_i = \frac{1}{|X_i|} \sum_{x \in X_i} x$$

4. Check for Convergence (usually if assignment in 2 does not change anymore)



[Bishop, Pattern Recognition]

How should the centers be initialized?

- naive solution: random sampling in X
- kmeans++

2.2 The k-means++ algorithm

We propose a specific way of choosing centers for the **k-means** algorithm. In particular, let $D(x)$ denote the shortest distance from a data point to the closest center we have already chosen. Then, we define the following algorithm, which we call **k-means++**.

1a. Take one center c_1, chosen uniformly at random from $\mathcal{X}$.
1b. Take a new center c_i, choosing $x \in \mathcal{X}$ with probability $\frac{D(x)^2}{\sum_{x \in \mathcal{X}} D(x)^2}$.
1c. Repeat Step 1b. until we have taken k centers altogether.
2-4. Proceed as with the standard **k-means** algorithm.

[Arthur, D., & Vassilvitskii, S. (2006). *k-means++: The advantages of careful seeding*]

k-Means can be approximated in Batches.

Algorithm 1 Mini-batch k-Means.

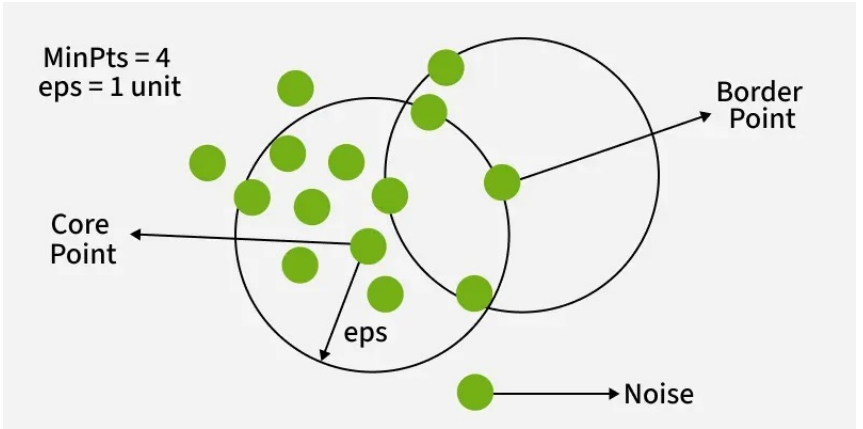
Given: k, mini-batch size b, iterations t, data set X
Initialize each $\mathbf{c} \in C$ with an $\mathbf{x}$ picked randomly from X
$\mathbf{v} \leftarrow 0$
for $i = 1$ to t **do**
  $M \leftarrow b$ examples picked randomly from X
  for $\mathbf{x} \in M$ **do**
    $\mathbf{d}[\mathbf{x}] \leftarrow f(C, \mathbf{x})$ // Cache the center nearest to $\mathbf{x}$
  end for
  for $\mathbf{x} \in M$ **do**
    $\mathbf{c} \leftarrow \mathbf{d}[\mathbf{x}]$ // Get cached center for this $\mathbf{x}$
    $\mathbf{v}[\mathbf{c}] \leftarrow \mathbf{v}[\mathbf{c}] + 1$ // Update per-center counts
    $\eta \leftarrow \frac{1}{\mathbf{v}[\mathbf{c}]}$ // Get per-center learning rate
    $\mathbf{c} \leftarrow (1 - \eta)\mathbf{c} + \eta\mathbf{x}$ // Take gradient step
  end for
end for

[Sculley, D. (2010). Web-scale k-means clustering]

DBSCAN: Clustering without known number of categories

Given: Affinity Matrix A, $\min_{\text{samples}}$ and ϵ

1. Define neighborhood matrix as $N = \{A \leq \epsilon\}$
2. For each sample, label it a *core* sample if $\geq \min_{\text{samples}}$ other samples within distance ϵ
3. Use depth-first search to find connected neighborhoods of core samples and their leaves
4. All other points are outliers



img source

k-Means

need to know the number of clusters

Requires well-defined $\mu_i = \frac{1}{|X_i|} \sum_{x \in X_i} x$

can run on minibatches

sensitive to k

outliers skew results

DBSCAN

automatic choice of clusters based on density

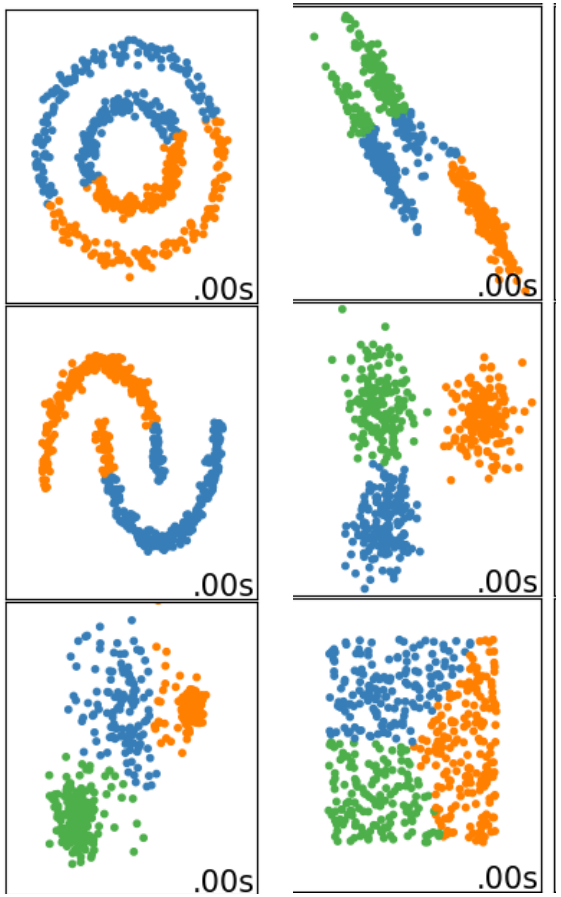
arbitrary distances, just requires neighborhood N

memory grows with n^2

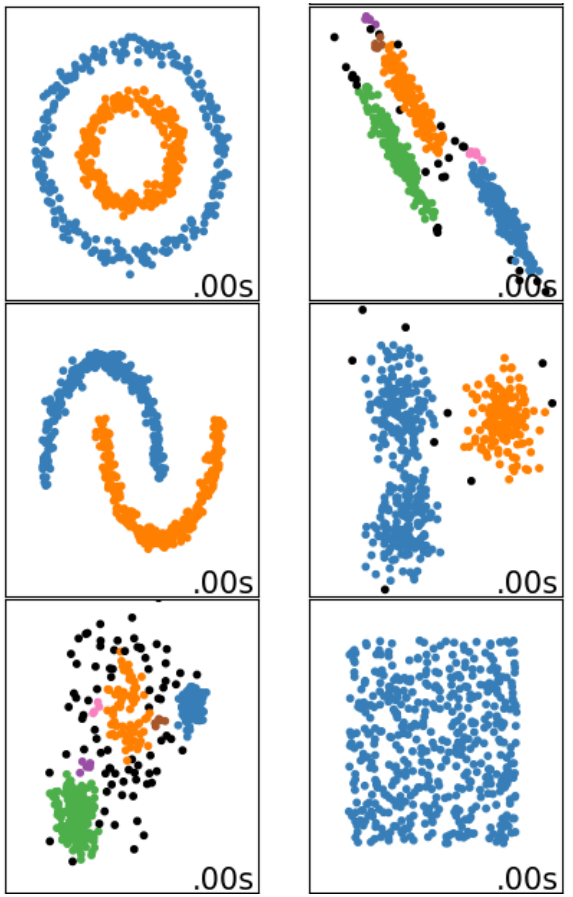
sensitive to ϵ

outliers are ignored

k-Means



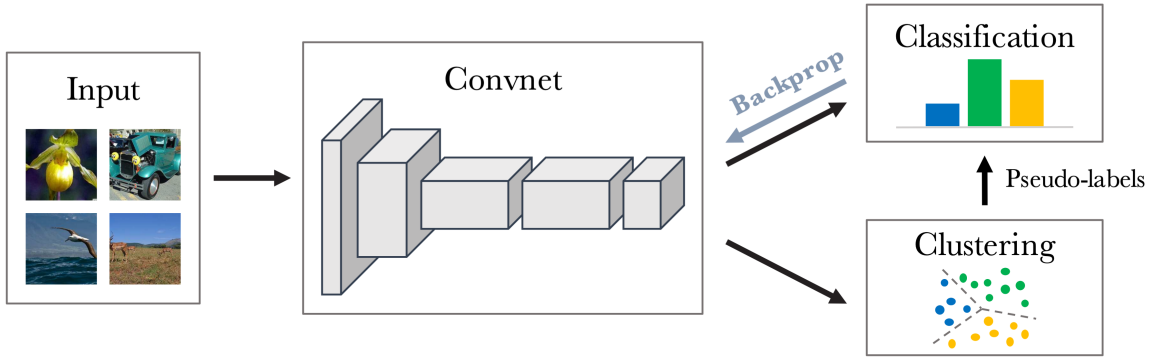
DBSCAN



What if I need DBSCAN but don‘t have memory?

- In some cases N is a sparse Matrix -> keep data in arrays instead of large matrix
- OPTICS is a more memory efficient variant (but usually slower)
- established trick in clustering: Reduce your dimensionality,
  e.g. in feature space through PCA,
  or in sampe space by subsampling or pre-clustering to get exemplars

Practical Example: Deep Clustering



Deep Clustering for Unsupervised Learning of Visual Features

Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze

Facebook AI Research
{mathilde,bojanowski,ajoulin,matthijs}@fb.com

Pratical Example: Deep Clustering

When θ is sampled from a Gaussian distribution, without any learning, f_θ does not produce good features. However the performance of such random features on standard transfer tasks, is far above the chance level. For example, a multilayer perceptron classifier on top of the last convolutional layer of a random AlexNet achieves 12% in accuracy on ImageNet while the chance is at 0.1% [42]. The good performance of random convnets is intimately tied to their convolutional structure which gives a strong prior on the input signal. The idea of this work is to exploit this weak signal to bootstrap the discriminative power of a convnet.

The essence of many SSL methods:
Anything better than chance can be exploited as a learning signal.

Iterative Clustering and Backprop

Clustering Step

1. extract features from images
2. reduce dimensionality through PCA
3. cluster with k-Means

Network Parameter Update

4. predict cluster-assignment for each image
5. backprop from cross-entropy

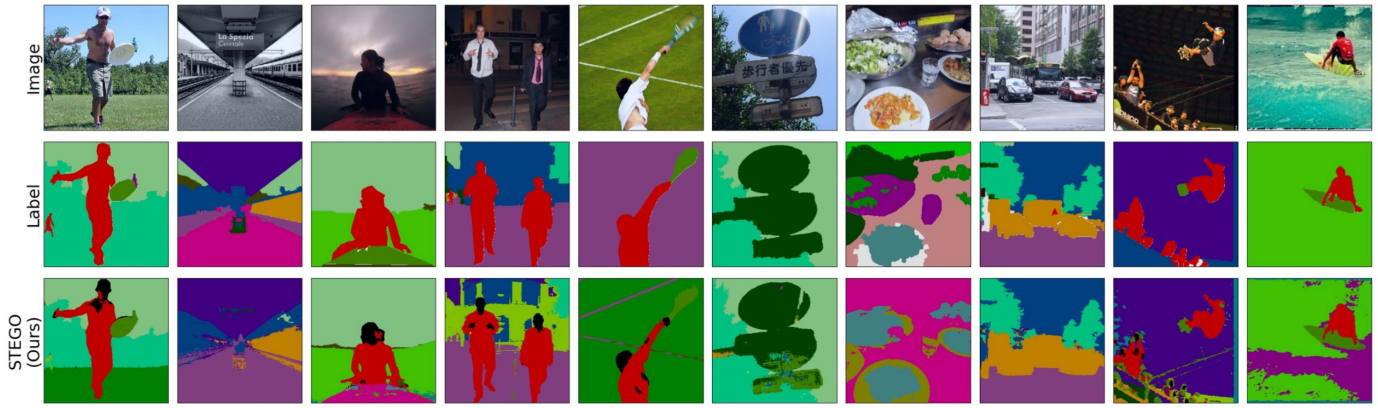
iterate every epoch

Pratical Example: Deep Clustering

Table 1: Linear classification on ImageNet and Places using activations from the convolutional layers of an AlexNet as features. We report classification accuracy averaged over 10 crops. Numbers for other methods are from Zhang *et al.* [72]

	ImageNet					Places				
Method	conv1	conv2	conv3	conv4	conv5	conv1	conv2	conv3	conv4	conv5
Places labels	–	–	–	–	–	22.1	35.1	40.2	43.3	44.6
ImageNet labels	19.3	36.3	44.2	48.3	50.5	22.7	34.8	38.4	39.4	38.7
Random	11.6	17.1	16.9	16.3	14.1	15.7	20.3	19.8	19.1	17.5
Pathak *et al.* [46]	14.1	20.7	21.0	19.8	15.5	18.2	23.2	23.4	21.9	18.4
Doersch *et al.* [13]	16.2	23.3	30.2	31.7	29.6	19.7	26.7	31.9	32.7	30.9
Zhang *et al.* [71]	12.5	24.5	30.4	31.5	30.3	16.0	25.7	29.6	30.3	29.7
Donahue *et al.* [15]	17.7	24.5	31.0	29.9	28.0	21.4	26.2	27.1	26.1	24.0
Noroozi and Favaro [42]	**18.2**	28.8	34.0	33.9	27.1	23.0	32.1	35.5	34.8	31.3
Noroozi *et al.* [43]	18.0	30.6	34.3	32.5	25.7	**23.3**	**33.9**	36.3	34.7	29.6
Zhang *et al.* [72]	17.7	29.3	35.4	35.2	32.8	21.3	30.7	34.0	34.1	32.5
DeepCluster	13.4	**32.3**	**41.0**	**39.6**	**38.2**	19.6	33.2	**39.2**	**39.8**	**34.7**

Pratical Example: Video Segmentation with STEGO



Published as a conference paper at ICLR 2022

UNSUPERVISED SEMANTIC SEGMENTATION BY DISTILLING FEATURE CORRESPONDENCES

Mark Hamilton
MIT, Microsoft
markth@mit.edu

Zhoutong Zhang
MIT

Bharath Hariharan
Cornell University

Noah Snavely
Cornell University, Google

William T. Freeman
MIT, Google

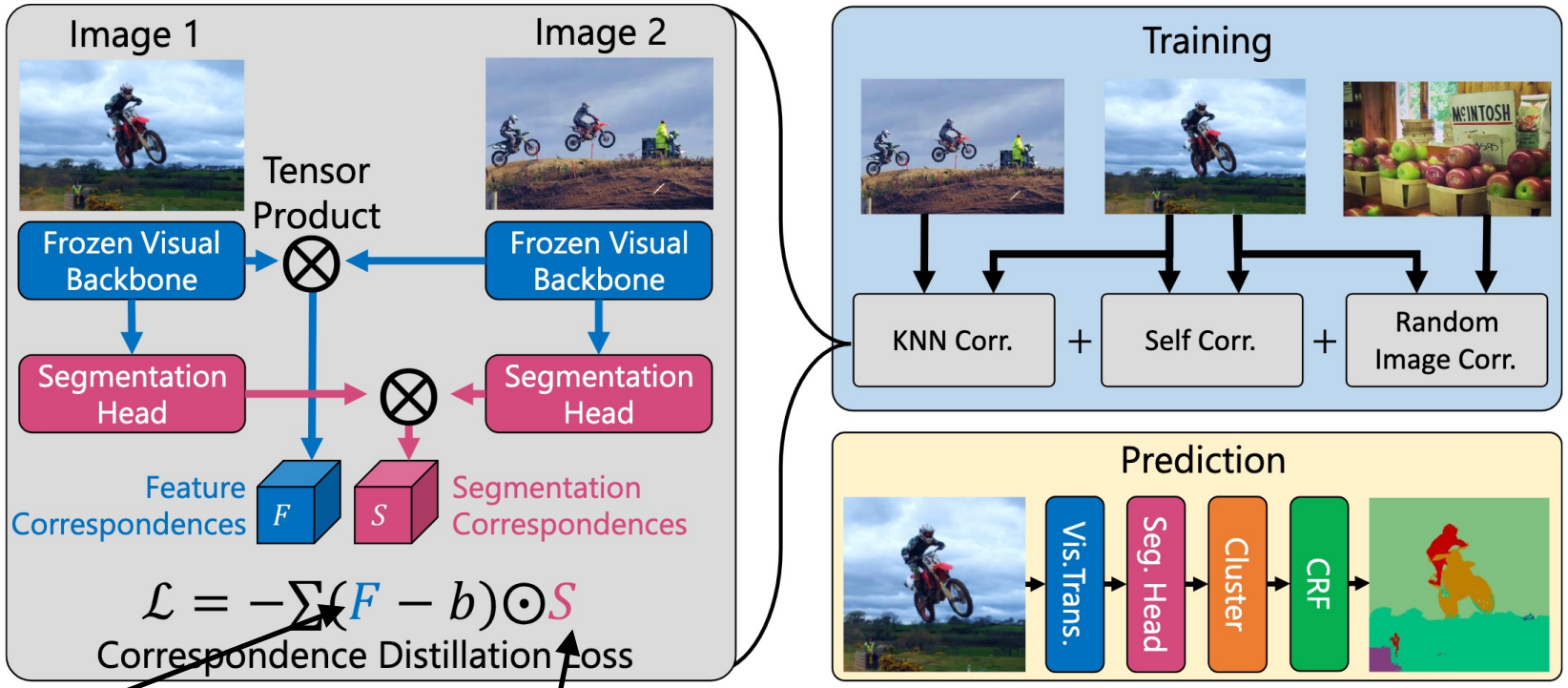
STEGO

Starts from the observation of feature correspondance:



How can this be refined for segmentation?

STEGO



feature correspondance between images f and g

also feature correspondance, but projected in lower dimension

$$F_{hwij} := \sum_c \frac{f_{chw}}{|f_{hw}|} \frac{g_{cij}}{|g_{ij}|},$$

Where b is a hyper-parameter which adds uniform "negative pressure" to the equation to prevent collapse. Minimizing $\mathcal{L}$ with respect to S encourages elements of S to be large when elements of $F - b$ are positive and small when elements of $F - b$ are negative. More explicitly, because the elements of F and S are cosine similarities, this exerts an attractive or repulsive force on pairs of segmentation features with strength proportional to their feature correspondences. We note that the elements of S are not just encouraged to *equal* the elements of F but rather to push to total anti-alignment (-1) or alignment (1) depending on the sign of $F - b$.

Learn well-clustering features, then cluster at inference

1. Sample images that (likely) contain similar classes
2. extract features from images
3. project them into a lower dimensional space
4. enhance similarity between projected points accross images



at inference:

5. cluster features
6. apply CRF

So what is a CRF?

CRF = Conditional Random Field

In segmentation, CRFs can optimize pixel-class assignment by giving the same label to close pixels of the same color.

$$E(\mathbf{x}) = \sum_i \psi_u(x_i) + \sum_{i<j} \psi_p(x_i, x_j),$$

overall energy of the segmentation x, we want to minimize this

cost of pixel i taking label x_i

e.g. inverse likelihood taken from softmax

pairwise cost of assigning labels x_i, x_j to pixels i, j
e.g. based on difference in color and position

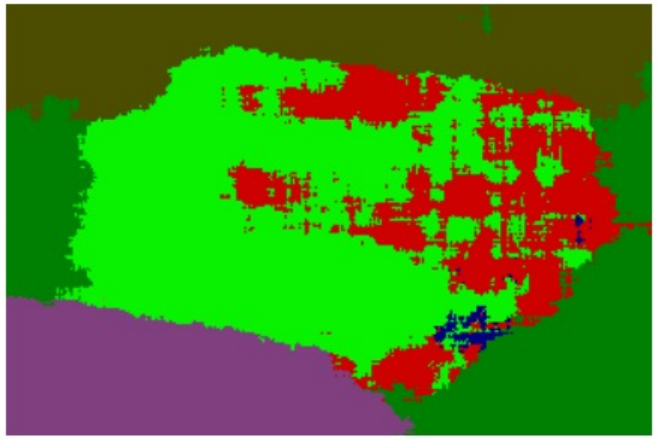
Krähenbühl, P., & Koltun, V. (2011). Efficient inference in fully connected crfs with gaussian edge potentials NeurIPS

So what is a CRF?

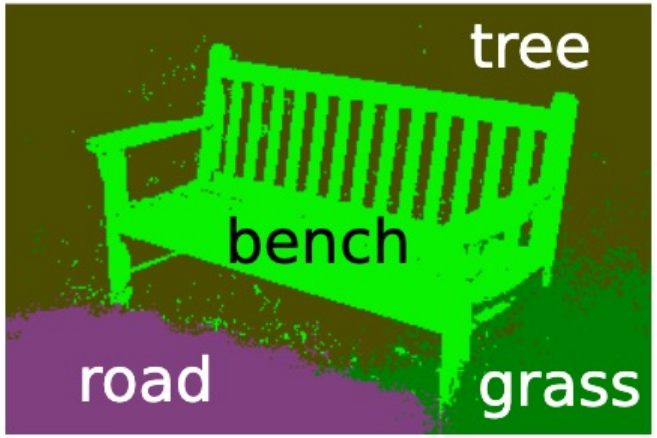
Image



Raw Assignment / Unary Potential



CRF output



Krähenbühl, P., & Koltun, V. (2011). Efficient inference in fully connected crfs with gaussian edge potentials NeurIPS

The STEGO objective optimizes a similar energy function

$$E(\phi) := \sum_{v_i, v_j \in \mathcal{V}} w(v_i, v_j)\mu(\phi(v_i), \phi(v_j))$$

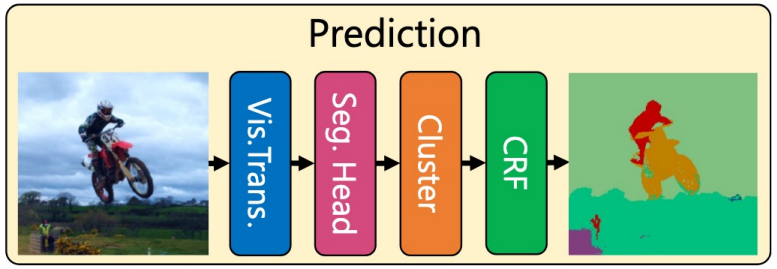
overall energy of the feature mapping

connectivity between features v
here the cosine distance

connectivity between mapped features $\phi(v)$
here the cosine distance

back to STEGO





Summary

- Nearest Neighbors
  - distance metrics
  - search structures
- Clustering
  - k-means
  - DBSCAN
- Deep Learning based Hybrids
  - Deep Clustering: use cluster assignment to refine features
  - STEGO: enhance feature similarity in similar images, cluster, CRF